ST445 · Week 3 · Managing and Visualising Data

Representing data with NumPy and Pandas

Dr Leonardo Alves Dias

Agenda

1. Introduction to File Handling
2. Introduction to NumPy
3. Introduction to Pandas



Learning Outcomes

By the end of this session, you should be able to:

1. Critically evaluate how structured, semi-structured and unstructured data are represented, from primitive data types and the international standards that govern them for a given analytical problem.

4. Design a reproducible data preprocessing workflow on real-world data, diagnosing and remedying common data quality problems, and critically justifying each decision taken.

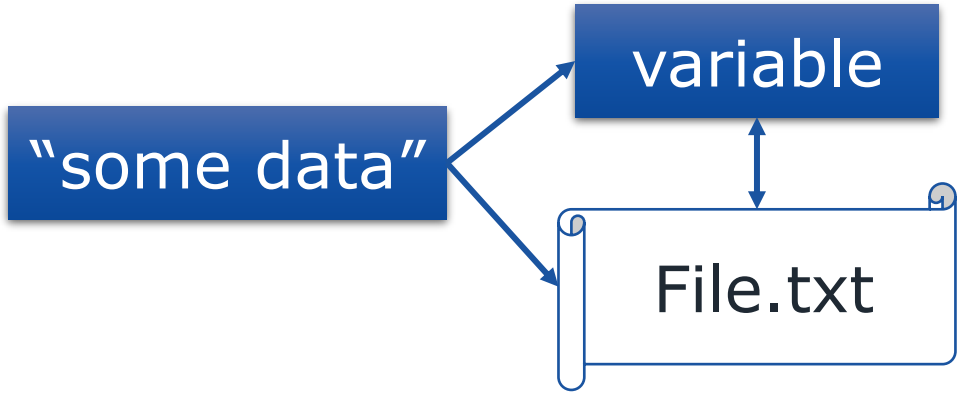


01

File Handling

Introduction

- File Handling refers to the procedure of storing data/information in a file with the help of a program. Thus, allowing us to preserve the data generated after we run the program.
  - A file is a computer resource for storing data (contiguous set of bytes), and it is stored in the secondary memory (non-volatile).



General Structure of a File

Usually, a file has the following structure:

- **Header:** metadata about the file's contents (file name, size, type, and so on).
- **Data:** contents of the file as written by the creator or editor.
- **End of file (EOF):** a special character that indicates the end of the file (In Python, it is an empty string).



[Reading and Writing Files in Python (Guide) – Real Python]

The open() function

- It is used to open a file and return a corresponding file object.
- Syntax: **open(**file, *mode*='r', *buffering*=-1, *encoding*=None, *errors*=None, *newline*=None, *closefd*=True, *opener*=None**)**

Modes:
r = read.
w = write.
a = append.
x = create.

```
f = open('example_file.txt')
```

```
f is a file object in read mode.
```

= The default mode for text files is read.

```
f = open('example_file.txt', 'rt')
```

```
f is a file object in read mode.
```

Built-in functions — Python 3.15.0 documentation

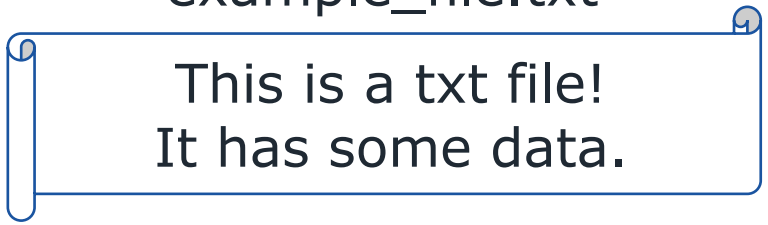
Reading files with open(): read() method

- To open files in different directories, specify the full address:

```
f = open('C:/Users/.../Lecture_examples/example_file.txt')
```

- The **read()** method returns the whole text:

example_file.txt




```
f = open('example_file.txt')
print(f.read())
```

```
This is a txt file!
It has some data.
```

7. Input and output — Python 3.15.0 documentation

Other Methods for Reading files with open()

example_file.txt

This is a txt file!
It has some data.

- **read()** also accepts a specific number of characters:

```
f = open('example_file.txt')
print(f.read(5))
```

```
This
```

- Read a single line (**readline()** method):

```
f = open('example_file.txt')
print(f.readline())
```

```
This is a txt file!
```

Methods for Reading files with open()

- **readlines()**: read all the lines as a list:

example_file.txt

This is a txt file!
It has some data.

```
f = open('example_file.txt')
print(f.readlines())
```

```
['This is a txt file!\n', 'It has some data.']
```

- Close a file:

```
f = open('example_file.txt')
f.close()
```

Closes a file.

Why Is It Important to Close Files in Python? – Real Python

Writing files with open()

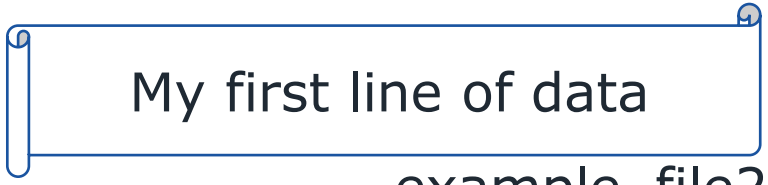
- To write in a file, open in writing (w) mode.

```
f = open('example_file2.txt', 'w')
```

If the file does not exist, it will be created. If the file exists, it will be opened, and the data will be overwritten.

- Use the **write()** method to send data to the file:

```
f = open('example_file2.txt', 'w')
try:
        f.write('My first line of data')
finally:
        f.close()
```



example_file2.txt

Guarantee that the close() is executed even if an error occurs.

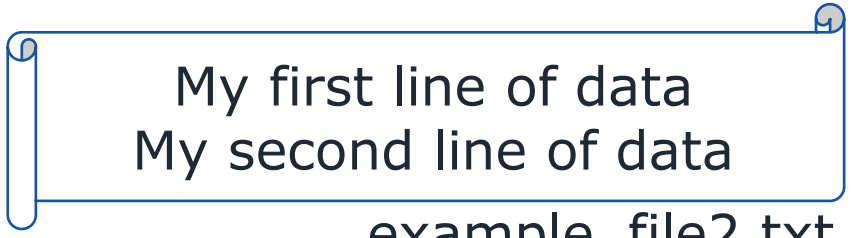
Appending to files with open()

- It is also possible to append data to existing files. Use mode = ‘a’.

```
f = open('example_file2.txt', 'a')
f.write('\nMy second line of data')
f.close()

f = open('example_file2.txt', 'r')
print(f.read())
f.close()
```

```
My first line of data
My second line of data
```



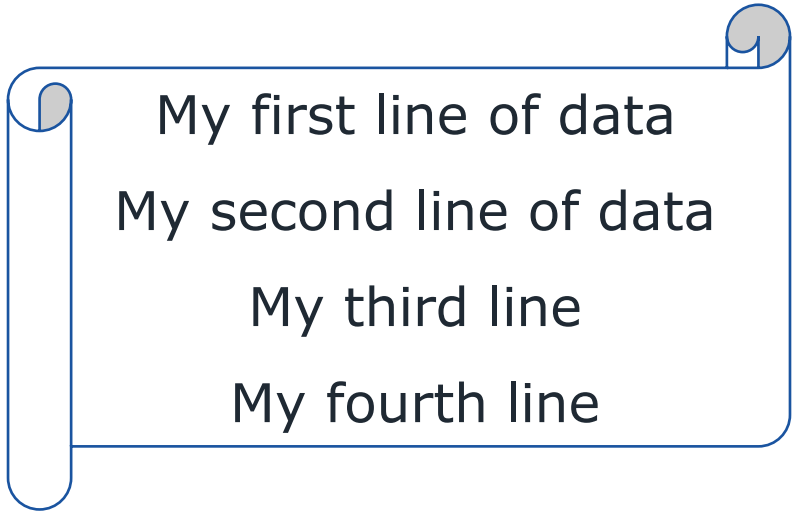
example_file2.txt

Appending to files with open()

```
f = open('example_file2.txt', 'a')
f.writelines(['\nMy third line', '\nMy fourth line'])
f.close()

f = open('example_file2.txt', 'r')
print(f.read())
f.close()
```

```
My first line of data
My second line of data
My third line
My fourth line
```



What if you just want to create a file?

- Use can use the mode = ‘x’ (Create).

```
f = open('example_file3.txt', 'x')
```

```
Directory:
example_file.txt
example_file2.txt
example_file3.txt
```

- Note that if the file already exists, an error occurs.

```
f = open('example_file3.txt', 'x')
```

```
---------------------------------------------------------------------------
--------
FileExistsError                           Traceback (most recent ca
ll last)
Input In [86], in <cell line: 1>()
----> 1 f = open('example_file3.txt', 'x')

FileExistsError: [Errno 17] File exists: 'example_file3.txt'
```

Using the “with … as” keyword pattern

- It is a good practice to deal with files using the “with as” keyword. The advantage is that the file is properly closed after its block of command lines finishes.

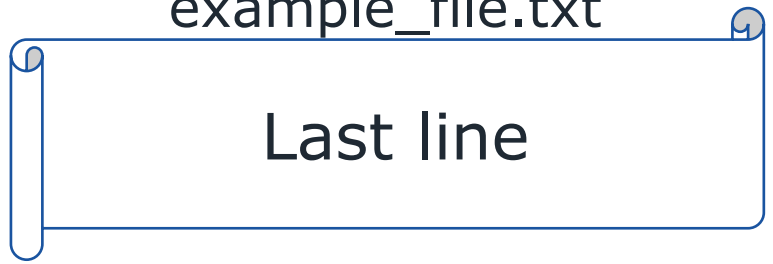
```
with open ('example_file.txt', 'r') as f:
      data = f.read()

#check if the file is closed.
print(f.closed)
```

```
True
```

```
with open ('example_file.txt', 'w') as f:
      data = 'Last line'
      f.write(data)
```

example_file.txt



02

Arrays and how numbers are stored

The NumPy ndarray

Arrays

- An array is a data structure that stores a collection of items under a single variable name.

- Python Arrays:

```
a = [1, "Hello", [3.14, "world"]]
a.append(2)
print(a)
```

```
[1, 'Hello', [3.14, 'world'], 2]
```

```
import array as arr
a = arr.array('i', [1, 2, 3])
print(a)
```

```
array('i', [1, 2, 3])
```

```
import numpy as np
a = np.array([1, 2, 3, 4])
print(a)
```

```
[1 2 3 4]
```

[Python Arrays - GeeksforGeeks]

One million decimal numbers

How much memory do they require to be stored as a Python list versus a NumPy array?

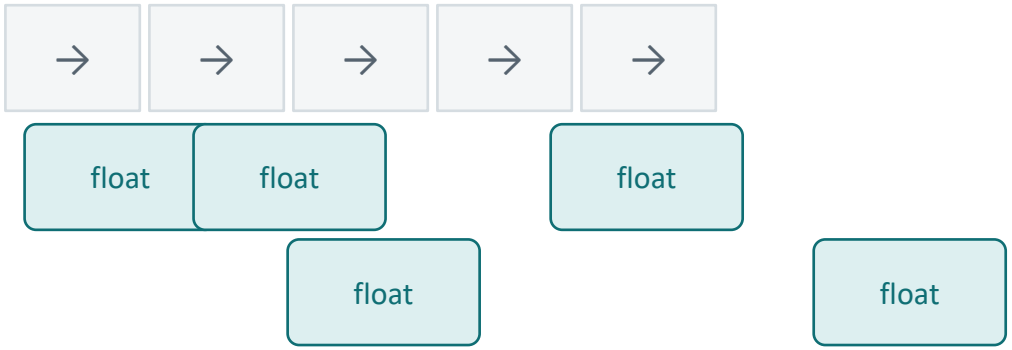
```
Python list: ____ MB
```

```
NumPy array: ____ MB
```

A list stores pointers; an array stores numbers

- A NumPy array keeps raw 8-byte values side by side in one contiguous block, whereas a list keeps pointers to separate objects.

Python list



Pointers (8 bytes each) to separate float objects (24 bytes each), anywhere in memory

NumPy array (float64)



The raw 8-byte values, side by side in one contiguous block

```
import sys, numpy as np
values = [float(i) for i in range(1_000_000)]
list_bytes = sys.getsizeof(values) + sum(sys.getsizeof(v) for v in values)
print(list_bytes / 1e6)                # 32.4 MB
print(np.array(values).nbytes / 1e6)   # 8.0 MB
```

sys.getsizeof does not count referenced objects, so they are added separately. Measured with Python 3.13 and NumPy 2.1.3.

[NumPy documentation: numpy.org/doc](numpy.org/doc)

Creating arrays with NumPy

- You'll make arrays a handful of ways. The most direct is `np.array()` from a Python list:

```
import numpy as np

a = np.array([1, 2, 3, 4])            # from a list → 1D array
b = np.array([[1, 2, 3], [4, 5, 6]]) # list of lists → 2D array (a matrix)

print(a)
print(b)
```

```
[1 2 3 4]
[[1 2 3]
 [4 5 6]]
```

Creating arrays with NumPy

- For ranges and grids, NumPy offers generators.
  - `arange` is like `range` but returns an array;
  - `linspace` provides a fixed *count* of evenly spaced points (great for plotting);
  - `zeros`, `ones`, and `full` build pre-filled arrays of a given shape:

```
import numpy as np
print(np.arange(0, 10, 2))   # start, stop (excluded), step → 0 2 4 6 8
print(np.linspace(0, 1, 5))  # 5 points evenly spaced from 0 to 1 inclusive
print(np.zeros(3))           # three zeros
print(np.ones((2, 3)))       # a 2×3 grid of ones
print(np.full((2, 2), 7))    # a 2×2 grid filled with 7
```

```
[0 2 4 6 8]
[0. 0.25 0.5 0.75 1. ]
[0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[[7 7]
 [7 7]]
```

The NumPy array (ndarray): a buffer plus metadata

A ndarray is a (usually fixed-size) multidimensional container of items of the same type and size.

Data buffer
A contiguous block of raw bytes

Dtype
How to read each element, e.g. float64: 8 bytes, IEEE 754

Shape
Number of elements along each axis, e.g. (3, 4)

Strides
Bytes to step to reach the next element along each axis

```
a = np.zeros((3, 4))
print(a)
a.dtype, a.shape, a.strides, a.nbytes
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
dtype('float64'), (3, 4), (32, 8), 96
```

Understanding Strides

- A tuple of byte steps used to navigate each dimension of a multi-dimensional array in memory.

Memory: one row of 12 values

0	1	2	3	4	5	6	7	8	9	10	11

```
a = np.arange(12, dtype=np.float64).reshape(3, 4)
a.strides                    # (32, 8)
a.T.strides                  # (8, 32): same buffer, new order
np.shares_memory(a, a.T)     # True: a view
```

shape (3, 4)

0	1	2	3
4	5	6	7
8	9	10	11

→ next column: +8 bytes

↓ next row: +32 bytes (4 × 8)

One dtype per array

- By default, a NumPy array has a one type only.

```
np.array([1, 2, 3]).dtype  # int64
np.array([1, 2.5]).dtype   # float64: the integer is converted
np.array([1, "a"]).dtype   # <U21: everything becomes text
np.array([1, "a"])[0] + 1  # error: "1" is now a string
```

```
import numpy as np
b = np.array([[1, 2, 3], [4, 5, 6]])
print("dtype:", b.dtype)
print("shape:", b.shape)
print("ndim: ", b.ndim)
print("size: ", b.size)
```

```
dtype: int64
shape: (2, 3)
ndim: 2
size: 6
```

Indexing and slicing

- Reading data out of an array starts the same as a Python list. First position starts at index 0 and negatives count from the end. Slices can be used obtained with *start:stop:step* (stop excluded):

```python
import numpy as np

a = np.array([10, 20, 30, 40, 50])

print(a[0])      # 10  -- first
print(a[-1])     # 50  -- last
print(a[1:4])    # [20 30 40] -- items 1,2,3
print(a[::2])    # [10 30 50] -- every other item
```

```python
import numpy as np
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(m[0, 2])    # 3  -- row 0, column 2
print(m[1])       # [4 5 6] -- the whole second row
print(m[:, 0])    # [1 4 7] -- the whole first column
print(m[0:2, 1:]) # top-right block: rows 0-1, columns 1+
```

Four kinds of indexing

Kind	Example	Selects	Returns
Basic	a[1, 2]	one element	a scalar
Slice	a[1:3, ::2]	a regular block	**a view**
Boolean mask	a[a > 5]	elements where the condition is True	**a copy, always 1-D**
Fancy	a[[0, 2]]	row indices 0 and 2, in that order	**a copy**

```
a = np.arange(12).reshape(3, 4)
a[1:3, ::2]      # array([[ 4,  6], [ 8, 10]])
a[a > 5]         # array([ 6,  7,  8,  9, 10, 11]): the 2-D shape is lost
a[[0, 2]].shape  # (2, 4)
```

NumPy documentation, "Indexing on ndarrays", numpy.org/doc/stable/user/basics.indexing.html.

Views and copies

- **View:** New shape or strides over the same buffer. No data copied.
- **Copy:** A new buffer holding its own data.

```
a = np.arange(12).reshape(3, 4)
np.shares_memory(a, a[1:3])      # True:  basic slice, a view
np.shares_memory(a, a.T)         # True:  transpose, a view
np.shares_memory(a, a[a > 5])    # False: boolean mask, a copy
np.shares_memory(a, a[[0, 2]])   # False: list of indices, a copy
```

NumPy documentation, "Copies and views", numpy.org/doc/stable/user/basics.copies.html.

Aliasing: changing a view changes the original

- **Useful:** Slicing a large array costs no memory and no time.
- **Dangerous:** A function that changes a slice it was given changes the caller's data.
- **Be careful:** if a function must not change its input, copy explicitly with .copy().

```
a = np.arange(12).reshape(3, 4)
b = a[:2]      # a view of the first two rows
b[0, 0] = 99
a[0, 0]        # 99: the original changed

c = a[:2].copy()  # an independent copy
c[0, 0] = -1
a[0, 0]           # still 99
```

NaN: a float that means "not a number"

```
x = np.array([1.0, np.nan, 3.0])
x.sum()           # nan: one NaN contaminates the result
np.nansum(x)      # 4.0: the nan-aware version skips it
np.nanmean(x)     # 2.0

np.nan == np.nan  # False: NaN equals nothing, not even itself
np.isnan(x)       # array([False,  True, False])

n = np.array([1, 2, 3])
n[0] = np.nan     # ValueError: cannot convert float NaN to integer
```

- **Be careful:** never test x == np.nan; it is always False. Use np.isnan(x).

IEEE Std 754-2019, section 6.2 (NaNs); NumPy documentation, "numpy.nan" and "numpy.nanmean".

Vectorisation and universal functions

- A *ufunc* is "a function that operates on ndarrays in an element-by-element fashion", also known as vectorisation.

A Python loop

```
out = []
for v in values:
    out.append(v * 2 + 1)
```

Each step: fetch an object, check its type, call the operation, create a new object.

A vectorised expression

```
out = a * 2 + 1  # a is an array
np.sqrt(a)       # a ufunc
np.add(a, 1)
```

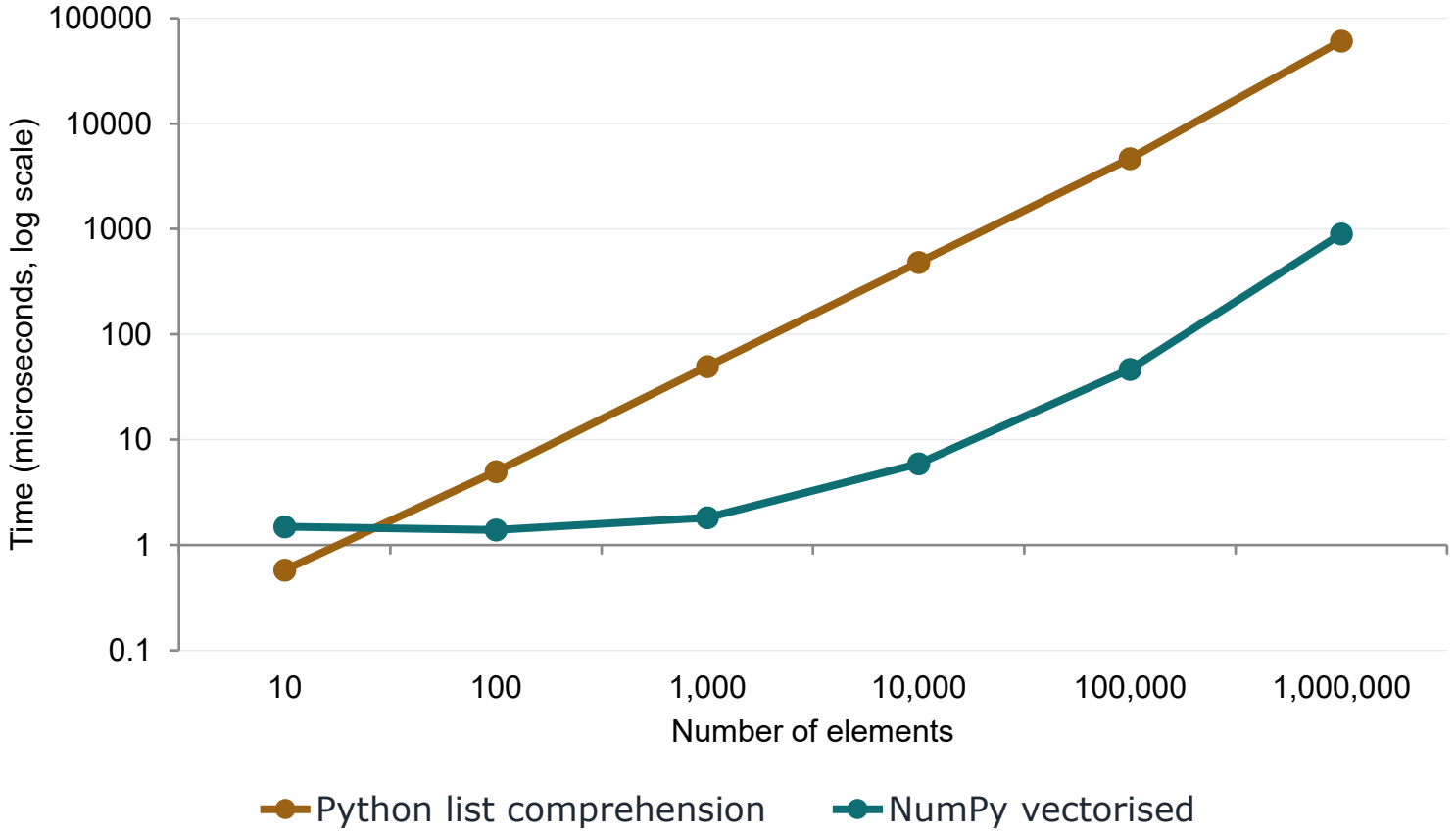
One compiled loop over the contiguous buffer, with the dtype known in advance.

Vectorised operations

```
import numpy as np
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
print(a + b)   # [11 22 33 44]
print(a * b)   # [10 40 90 160]
print(a / b)   # [10. 10. 10. 10.]
print(a ** 2)  # [100 400 900 1600]
```


NumPy documentation, "Universal functions (ufunc)", numpy.org/doc/stable/reference/ufuncs.html; Harris et al. (2020).

How much faster? It depends on the size



10 elements

The loop is faster: NumPy's overhead dominates

1,000 or more

NumPy is about 25 to 100 times faster

Your machine

Different numbers, same shape: measure

*Measured with timeit (best of 5 repeats), Python 3.12, NumPy 2.4. Expression: x * 2 + 1.*

Benchmark, do not guess

```
values = list(range(1_000_000))
a = np.arange(1_000_000)

%timeit [v * 2 + 1 for v in values]
# example output: 120 ms ± 26.3 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

%timeit a * 2 + 1
# 3.02 ms ± 104 us per loop
```

- **Be careful:** vectorised code creates temporary arrays as large as the input, so it trades memory for speed. Readable code that is fast enough is better than clever code that is slightly faster.

Time for a short break

We will resume in 5 minutes

Rest your eyes, stretch and grab some water.



Broadcasting: math between different shapes

- Broadcasting in NumPy allows us to perform arithmetic operations on arrays of different shapes without reshaping them.
- It automatically adjusts the smaller array to match the larger array's shape by replicating its values along the necessary dimensions.

```python
import numpy as np

a = np.array([1, 2, 3, 4])
# an operation between an array and a scalar
print(a + 100)    # 100 is added to every element → [101 102 103 104]
print(a * 0.5)    # every element halved
```

```
[101 102 103 104]
[0.5 1. 1.5 2. ]
```

Broadcasting — NumPy v2.5 Manual

Broadcasting: the rule

1. Compares the shape element-wise, starting with the trailing (i.e. rightmost) dimension and works its way left.

2. Two dimensions are compatible if they are equal, or if one of them is 1.

3. A missing dimension counts as 1. The result takes the larger size in each dimension.

Shape of a	Shape of b	Result	Why
(3, 4)	(4,)	**(3, 4)**	b counts as (1, 4) and is repeated down 3 rows
(3, 1)	(4,)	**(3, 4)**	both are stretched
(3, 4)	(3,)	**error**	4 and 3 differ, and neither is 1

NumPy documentation, "Broadcasting", numpy.org/doc/stable/user/basics.broadcasting.html.

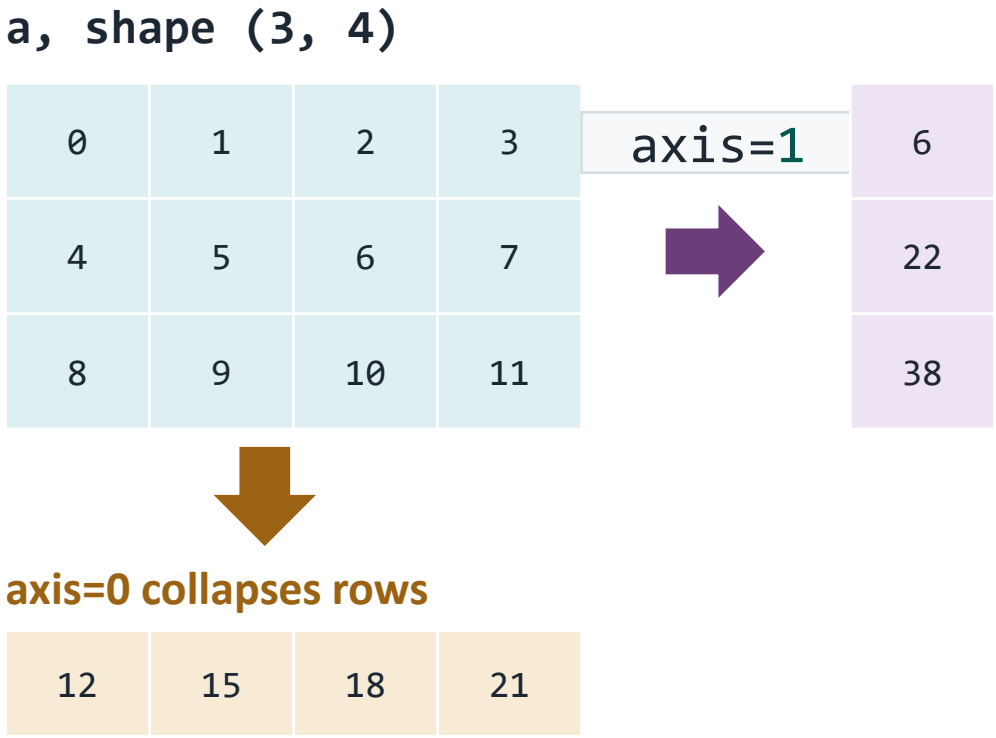
Broadcasting, pictured




```
col = np.array([[0], [10], [20]])  # shape (3, 1)
row = np.array([1, 2, 3, 4])       # shape (4,)
col + row
```

```
array([[ 1,  2,  3,  4],
       [11, 12, 13, 14],
       [21, 22, 23, 24]]) # shape (3, 4)
```

Aggregations along an axis




```
a = np.arange(12).reshape(3, 4)

# rows disappear: shape (4,)
a.sum(axis=0)  # array([12, 15, 18, 21])

# columns disappear: shape (3,)
a.sum(axis=1)  # array([ 6, 22, 38])
```

- **Be careful:** axis=0 combine rows and axis=1 combines columns.

Reductions, and keepdims

```
a = np.arange(12).reshape(3, 4).astype(float)

a.max(axis=1)                                  # array([ 3.,  7., 11.])
a.argmax(axis=1)  # array([3, 3, 3]): position of each row's maximum
(a > 5).any(axis=1)                            # array([False,  True,  True])

row_means = a.mean(axis=1, keepdims=True)  # shape (3, 1), not (3,)
centred = a - row_means                    # broadcasts: each row now has mean 0
centred[0]                                 # array([-1.5, -0.5,  0.5,  1.5])
```

- **Be careful:** a reduction over a NaN returns NaN (slide on NaN). Use the nan-aware versions, such as np.nanmax, when skipping is intended.

Random numbers: the Generator API and seeds

```
rng = np.random.default_rng(445)
rng.normal(size=3) # array([-0.076,  0.252,  0.931])
rng.integers(1, 7, size=5)  # dice rolls
rng.choice(["GBR", "FRA", "DEU"], size=2,
           replace=False)
```

```
[-0.07618361 0.25246565 0.93091556]
[4 3 4 3 5]
['FRA' 'DEU']
```

- **Be careful:** a seed makes results repeatable, not representative. When randomness matters, report results across several seeds, and record every seed with the code.

NumPy documentation, "Random Generator", numpy.org/doc/stable/reference/random/generator.html; NumPy Enhancement Proposal 19 (2018).

03

Pandas

Introduction

- pandas is an open-source library providing high-performance, easy-to-use data structures and data analysis tools.

- pandas is often used to represent tabular data as DataFrames.

- In many cases, DataFrames are faster, easier to use, and more powerful than tables.

```
import pandas
```

pandas documentation — pandas 2.2.3 documentation

Creating a Pandas Series

- The Pandas Series is a one-dimensional labeled array capable of holding any data type.

```
import pandas

myarray = [3, 7, 2]

myvar = pandas.Series(myarray)
print(myvar)
print(myvar[0])
```

```
0    3
1    7
2    2
dtype: int64
3
```

- Note that pandas index every array value automatically, starting from index 0.

pandas documentation, "Intro to data structures", pandas.pydata.org/docs/user_guide/dsintro.html; McKinney (2017), chapter 5.

Creating a Pandas Series

- Defining labels for indexes.

```
import pandas

myarray = [3, 7, 2]

myvar = pandas.Series(myarray, index = ["x", "y", "z"])
print(myvar)
print(myvar["x"])
```

```
x    3
y    7
z    2
dtype: int64
3
```

Creating a Pandas DataFrame

The Pandas DataFrame is a 2-dimensional labeled data structure with columns of potentially different types.

```
import pandas

mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}
myvar = pandas.DataFrame(mydataset)
print(myvar)
```

	cars	passings
0	BMW	3
1	Volvo	7
2	Ford	2

- Note: this is similar to a dictionary but with multiple values for the same key.

[The pandas DataFrame: Make Working With Data Delightful]

Creating a Pandas DataFrame

- Renaming the indexes with personal labels.

```
import pandas

mydataset = {
    'cars': ["BMW", "Volvo", "Ford"],
    'passings': [3, 7, 2]
}
myvar = pandas.DataFrame(mydataset, index = ["c1", "c2", "c3"])
print(myvar)
```

	cars	passings
c1	BMW	3
c2	Volvo	7
c3	Ford	2

The index: a key and an alignment device

- **A key:** Look a row up by its label: $\text{df.loc}[(\text{"GBR"}, 2019)]$
- **An alignment device:** When two objects are combined, rows with the same label meet

```
df = pd.DataFrame({"iso3": ["GBR", "FRA", "DEU"],
                   "year": [2019, 2019, 2019],
                   "pop_m": [66.8, 67.4, 83.1]})
df = df.set_index(["iso3", "year"])  # a two-level index: the Week 2 join key
df.loc[("GBR", 2019), "pop_m"]       # 66.8
df.index.is_unique                   # True: safe to use as a key
```

	iso3	year	pop_m
0	GBR	2019	66.8
1	FRA	2019	67.4
2	DEU	2019	83.1

		pop_m
iso3	year	
GBR	2019	66.8
FRA	2019	67.4
DEU	2019	83.1

Be careful: pandas allows duplicate index labels. A lookup then returns several rows, and alignment can multiply them. Check df.index.is_unique whenever the index is meant to be a key.

pandas documentation, "MultiIndex / advanced indexing", pandas.pydata.org/docs/user_guide/advanced.html.

Selecting rows and columns

Method	Selects	Example
.loc	label	df.loc["b":"d", ["x", "y"]]
.iloc	integer position	df.iloc[0:2, 0]
boolean mask	condition	df[df["x"] > 0]
.query	expression	df.query("x > 0 and y == 1")

```
df = pd.DataFrame({"x": [1, -2, 3, 4], "y": [0, 0, 1, 1]}, index=list("abcd"))
df.loc[df["x"] > 0, "y"]      # rows by condition, one column by label
df.query("x > 0 and y == 1")  # rows c and d
```

```
   x  y
a  1  0
b -2  0
c  3  1
d  4  1
```

pandas documentation, "Indexing and selecting data", pandas.pydata.org/docs/user_guide/indexing.html.

Access a DataFrame index

- Analyse the following 4x3 DataFrame (4 rows and 3 columns):

```
import pandas as pd
technologies = {
    'Courses' : ["Spark", "PySpark", "Hadoop", "Python"],
    'Fee' : [20000,25000,26000,22000],
    'Duration' : ['30days', '40days', '35days', None]
    }
index_labels=['r1','r2','r3','r4']
df = pd.DataFrame(technologies,index=index_labels)
print(df)
```

	Courses	Fee	Duration
r1	Spark	20000	30days
r2	PySpark	25000	40days
r3	Hadoop	26000	35days
r4	Python	22000	None

Access a DataFrame column

- You can provide the column label(s).

	Courses	Fee	Duration
r1	Spark	20000	30days
r2	PySpark	25000	40days
r3	Hadoop	26000	35days
r4	Python	22000	None

```
df2 = df['Courses']
# Select one column by Index Label
print(df2)
```

```
r1    Spark
r2    PySpark
r3    Hadoop
r4    Python
Name: Courses, dtype: object
```

```
df2 = df[['Courses', 'Duration']]
# Select columns by Index Label List
print(df2)
```

```
  Courses  Duration
r1   Spark    30days
r2  PySpark  40days
r3  Hadoop   35days
r4  Python    None
```

Access a DataFrame row

- You can provide one or more rows using their index. Use **.loc()** to access the index values or **.iloc()** for the index's respective integer value.

```
df2 = df.iloc[2]     # Select Row by Index
df2 = df.iloc[[2,3,6]]    # Select Rows by
Index List
df2 = df.iloc[1:5]   # Select Rows by
Integer Index Range
df2 = df.iloc[:1]    # Select First Row
df2 = df.iloc[:3]    # Select First 3 Rows
df2 = df.iloc[-1:]   # Select Last Row
df2 = df.iloc[-3:]   # Select Last 3 Row
df2 = df.iloc[::2] # Selects alternate rows
```

```
# Select Row by Index Label
df2 = df.loc['r2']
# Select Rows by Index Label List
df2 = df.loc[['r2','r3', 'r4']]
# Select Rows by Label Index Range
df2 = df.loc['r1':'r4']
# Select Rows by Label Index Range
df2 = df.loc['r1':'r4']
# Select Alternate Rows with in Index
Labels
df2 = df.loc['r1':'r4':2]
```

Access a DataFrame element

- You can provide the row and column label(s) using the **.at** method (or its equivalent integer indexes using **.iat**).

```
df2 = df.at['r3','Duration']
# Select one element based on the [row, column] Index Label
print(df2)
```

```
35days
```

```
df2 = df.iat[2, 2]
# Select one element based on the [row, column] Integer Index Label
print(df2)
```

```
35days
```

Reading a .csv file with pandas

- You can read a CSV (comma-separated file) using the **.read_csv()** function.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df)
```

```
     Duration  Pulse  Maxpulse  Calories
0          60    110       130     409.1
1          60    117       145     479.0
2          60    103       135     340.0
3          45    109       175     282.4
4          45    117       148     406.0
..        ...    ...       ...       ...
164        60    105       140     290.8
165        60    110       145     300.0
166        60    115       145     310.2
167        75    120       150     320.4
168        75    125       150     330.4

[169 rows x 4 columns]
```

Summary



We introduce File Handling with open() and with open()



We introduce NumPy, indexing and broadcasting.



We introduce Pandas Series and DataFrame.



We introduce Pandas Indexing.

Thank you

Questions?

 l.alves-dias@lse.ac.uk

 linkedin.com/school/lse-department-of-statistics/

 @londonschoolofeconomics

 github.com/LeonardooAlves/LSE-ST445